

Cloud Computing

Submission Deadline: April 17th 2025 (Friday) **Optional Check-in by March 27th**

Form a Team of 1 to 2 members and select only one single topic below. Read the following.

- 1) Each team submits to the NTU Learn submission link a **single Zip File with the Final Report and PPT Powerpoint Presentation Slides** with all names/student IDs.
- 2) NO Page Limit in Final Report—typically has an introduction, literature review, problem statement, solution design, illustrative examples, conclusions, references. and computer software in the Zip file or open-source repository, e.g., Github, for my reference.
- 3) Level of work is expected to commensurate with the team size. A project demonstrating originality and creativity can get higher marks. Give your best!
- 4) Teams may be invited to give short optional demos to share their work. See optional check-in procedures in course announcement.

Start early. Be Imaginative. Creative Tool-Wielding. Have Fun!

Topic 1: Future Data Centers and Diagram-as-Code

This project investigates the evolving landscape of AI workloads in future data centers, focusing on their computational, storage, and energy demands. As AI models grow in complexity and scale, earth-based data centers must adapt to handle tasks like training large language models (LLMs) and real-time inference. Now, data centers are reimagined through initiatives like StarCloud (see <https://x.com/karpathy/status/1998806260783919434>) and SpaceX's Solar Data Centers, which conceptualizes a future where orbital space-borne data centers scale up AI workload and reduce terrestrial energy burden with latency-optimized networking (Cloud 3.0).

- (i) Analyze Cloud 3.0 data center topologies and networking by proposing how earth-based and space-borne data centers can be managed by topology design, resource allocation, scheduling of cloud hardware like GPUs, TPUs, supercomputers. One possibility is a hybrid of optical circuit (laser) and packet switching architecture, e.g., Helios: <https://cseweb.ucsd.edu/~vahdat/papers/helios-sigcomm10.pdf>.
- (ii) Explore **Diagram-as-Code (DaC)** as a Visualization Tool to illustrate data center topologies. See examples of AWS cloud architecture using the Mermaid software at: <https://medium.com/@koshea-il/architecture-diagrams-as-code-mermaid-vs-architecture-as-code-d7f200842712>. Can Gen-AI generate the right 'Diagram code'?
- (iii) Propose how topology can become a programmable entity so that one can declare the computational intent—for example, "orchestrate a 100-GPU training job with low Collective Completion Time (CCT) and high GPU utilization"—in a structured code format. A generative AI layer would then interpret this intent to produce and validate an optimal *logical* network diagram in a tool like Mermaid. This AI-generated DaC, rather than a human-drawn sketch, can then dynamically reconfigure switches and optical links, moulding the physical topology to fit the workload's needs. This creates a closed loop where the infrastructure is as fluid as the software it runs. See https://web.ee.technion.ac.il/~isaac/p/nines26_dcSim.pdf, <https://github.com/LLNL/TreeScope> & https://github.com/notYOrick/matplotlib_ai.

Topic 2: Personal Assistant-as-a-Service

OpenClaw (initially Clawdbot then renamed Moltbolt later renamed OpenClaw) is the hottest open-source AI Agent project of 2026, boasting over 100,000 GitHub stars. It reads your email. It can complete daily tasks like booking flights. It stores your API keys. It builds an evolving model of your habits and preferences over time. That's what makes it powerful, and already many cloud computing companies are offering OpenClaw-as-a-Service, e.g., Alibaba Cloud: <https://www.alibabacloud.com/en/campaign/ai-openclaw> and a few others: https://www.theregister.com/2026/02/04/cloud_hosted_openclaw.

Explore the principle of Personal Assistant (PA)-as-a-Service and build a software platform where PA capabilities are decomposed into independent services—like scheduling, and task management that communicate via well-defined RESTful API calls. Demonstrate novel use of different APIs using AI agents with illustrative examples. For example, PA uses Mastodon API to fetch a user's feed and analyze trends. For instance, PA uses Github API to 'vibe-code', showing an API-driven development cycle from data processing to frontend user presentation.

Topic 3: Cloud Security I: Fuzzing in the Cloud

This project leverages cloud computing to execute large-scale fuzzing to test the security of systems in the cloud. Microsoft researchers initiated the idea of fuzzing in the cloud in 2010:

<https://www.microsoft.com/en-us/research/wp-content/uploads/2016/02/paper-80.pdf>

By 2016, as the Open-Source Software (OSS) movement took root and cloud systems were increasingly based on OSS, Google researchers started exploring AI-powered fuzzing:

<https://security.googleblog.com/2023/08/ai-powered-fuzzing-breaking-bug-hunting.html> and paper: <https://arxiv.org/pdf/2510.02185>.

- (i) Start by exploring the Cloudflare's guide to Linux kernel fuzzing available here: <https://blog.cloudflare.com/a-gentle-introduction-to-linux-kernel-fuzzing/> and understanding the use of a fuzzing tool called American Fuzzy Lop fuzzer (<https://lcamtuf.coredump.cx/afl>).
- (ii) Having understood the basics, you can choose to fuzz either a network protocol (e.g., Linux TCP/IP networking stack, HTTP/2, QUIC in load balancers) or internal cloud computing protocols (e.g., Kubernetes and cloud orchestration tools) to discover vulnerabilities in systems-level code. Discuss how to scale up automatically fuzzing workloads on elastic cloud resources, accelerating the discovery of vulnerabilities in the many OSS platforms that constitute modern cloud infrastructure.

Topic 4: Cloud Security II: AI Worms

This project investigates the security threat of AI "prompt worms" in networks of autonomous agents, as exemplified by the OpenClaw and Moltbook ecosystem. See this blog article:

<https://arstechnica.com/ai/2026/02/the-rise-of-moltbook-suggests-viral-ai-prompts-may-be-the-next-big-security-threat/>. These threats exploit the core function of AI agents—following

instructions—by using self-replicating prompts to spread harmful directives. Due in part to careless use of vibe coding, such a worm could propagate through agent communication

channels (social feeds, emails), leading to data exfiltration, spam and system compromise.

- (i) Aim to dissect this novel attack vector by analyzing the propagation mechanisms across interconnected, memory-persistent agents and the "lethal trifecta" of vulnerabilities (data access, untrusted content, external communication).
- (ii) Develop and evaluate effective containment strategies before this AI worm threat escalates as technology shifts from centrally controlled API-based models to decentralized, locally-run agents. Assess technical countermeasures like API pattern monitoring and propose security frameworks to mitigate risks.

Topic 5: Generative Information Retrieval for Science

This project addresses a critical problem in AI for science: Can we trust the answers from AI research tools? Generative search engines and AI research assistants promise to find and synthesize information from scientific sources, but they often show weak or misleadingly wrong citations. First, develop a method to improve generative information retrieval of scientific information that can improve citation accuracy. This may require solving the *provenance problem*: a systematic breakdown in the chain of scholarly acknowledgement that current ethical frameworks fail to address. Finally, create a SaaS tool for scientific information retrieval.

1. *Open-source AI tool beats giant LLMs in literature reviews — and gets citations right*, E. Gibney, Nature, Feb. 2026: <https://www.nature.com/articles/d41586-026-00347-9>
2. *LLM use in scholarly writing poses a provenance problem*, Brian D. Earp et al, *Nat Mach Intell* **7**, 1889–1890, 2025, <https://www.nature.com/articles/s42256-025-01159-8>
3. *DeepTRACE: Auditing Deep Research AI Systems for Tracking Reliability Across Citations and Evidence*, Pranav Narayanan Venkit et al, 2026 <https://arxiv.org/abs/2509.04499>

Topic 6: SaaS-based Agentic Engineering

In this project, you explore LLM agent programming using an Agentic AI SaaS called Nemobot (<https://nemobot-neue-experiment.vercel.app>), which allows exploring tool-augmented generation and the design of programmable prompts systematically using natural language.

- (i) Walk through the Nemobot tutorial at: <https://intii.craft.me/ICZZhMWH18GUf4>
- (ii) Study *prompt decomposition* or *prompt differentiation*—links below—via Nemobot.
- (iii) Nemobot supports agent programming with local tools, i.e., tool-augmented generation. Design an application in Nemobot using tool-augmented generation or remote *Agent Skills* (<https://developers.openai.com/codex/skills>).
- (iv) While vibe-coding is when humans prompt AI to write code, OpenAI cofounder Andrej Karpathy recently said ‘*agentic engineering*’ is when AI agents write the code themselves. Attempt ‘*agentic engineering*’ in Nemobot.

1. Prompt decomposition: *Decomposed Prompting: A Modular Approach for Solving Complex Tasks*, Tushar Khot et al, ICLR 2023: <https://arxiv.org/abs/2210.02406>
2. Prompt differentiation: *TextGrad: Automatic "Differentiation" via Text*, Mert Yuksekgonul et al, Nature, 2024 <https://arxiv.org/abs/2406.07496>

Topic 7: Coding-as-a-Service

Major AI companies are rolling out Coding-as-a Service tools like Cursor's Cloud Agent, OpenAI Codex Web, Github Copilot, Anthropic Claude on the Web, Google's Gemini Code Assist. These Coding-as-a-Service tools use LLMs for code generation, bug detection or enable vibe coding. This project explores how to make Coding AI agents and creating a Coding-as-a-Service tool that enhances user experience, accuracy, productivity, safety, and performance.

- (i) Read UC Berkeley's Ben Recht blog article on asking Claude Coder to build itself: <https://www.argmin.net/p/secrets-of-intelligence-services>. Study the open-sourced code at: <https://github.com/benjamin-recht/microagent>, and describe in your own words with visualization how an AI coding agent works in general with API calls.
- (ii) Extend the above Claude Coder or design your own into a Coding-as-a-Service.
- (iii) Describe how you evaluate your Coding-as-a-Service using performance metrics (e.g., Pass@k, CodeBLEU, functional correctness), see OpenAI paper "Evaluating Large Language Models Trained on Code" (<https://arxiv.org/abs/2107.03374>).
- (iv) Show how your Coding-as-a-Service is useful to a SaaS user, e.g., allow someone to vibe-code, teach junior programmers to write code, do scientific computing etc.

Topic 8: X-as-a-Service

This project studies how to design and implement a "X-as-a-Service" (XaaS) solution, turning complex capabilities into scalable, on-demand cloud services. First, identify a challenging problem of your interests that can be addressed via Software-as-a-Service. You may consult https://en.wikipedia.org/wiki/As_a_service to identify a high-value "X"—a specific function, process, or expertise that is currently difficult, expensive, or slow to access. If you wish, use the principle of crowdsourcing, which involves architecting this "X" as a dual-utility cloud service in "killing two birds with one stone". This means intentionally intertwines two distinct workflows—one potentially commercial or operational, the other research-driven or societal—ensuring that every unit of contributed human or computational effort yields value for two separate objectives simultaneously. Then, develop a AI-powered solution for your XaaS.

Here are some examples. To combat disinformation from *Propaganda-as-a-Service* (<https://arxiv.org/pdf/2112.05224>), a *Factchecker-as-a-Service* would go beyond single claim verification—it can also detect viral falsehoods and forecast their real-world impact. Another example is ***Scientific Computing-as-a-Service*** as exemplified by Google's Pi Delivery (<https://pi.delivery>), which creates a simple API to allow anyone to fetch a specific sequence of π . While the service handles the immense backend challenge of processing trillions of digits to determine accurate π , it unlocks powerful applications, e.g., a scientist could statistically analyze digit distributions for number theory research, a musician or digital artist could use the same API stream to generate algorithmic compositions or seed procedurally-generated visual art, democratizing computational power for both rigorous science and creative exploration etc.